

WelfareBuddy – Rendszerfelépítés

Bevezetés

A jelen dokumentum a WelfareBuddy szoftverrendszer technikai felépítését, komponenseit, kommunikációs csatornáit, adatfolyamatait, biztonsági intézkedéseit és üzemeltetési topológiáját írja le.

Tartalomjegyzék

Bevezetés.....	1
1. Áttekintő architektúra	3
2. Technológiai stack	3
2.1 Backend	3
2.2 Frontend (web + mobil közös kódbázis).....	3
2.3 Mobil shell és pluginek (csak mobil/)	4
3. Backend (REST API).....	4
3.1 Általános jellemzők.....	4
3.2 Végpontok (api.php).....	4
3.3 Batch feltöltési szabály	8
3.4 Streak frissítés	8
4. Valós idejű kommunikáció (WebSocket).....	8
4.1 Technológia	8
4.2 Csatornák és események	9
5. Frontend architektúra	9
5.1 Projekt struktúra.....	9
5.2 AppShell és provide / inject.....	10
5.3 Offline kezelés	10
6. Egészségügyi aggregátor integráció	10
6.1 Támogatott adattípusok.....	10
6.2 Szinkronizációs folyamat	10
6.3 Hibakezelés.....	11
7. Üzemeltetés és deployment.....	11

7.1 Környezetek.....	11
7.2 Tárhely és háttér szolgáltatások.....	11
7.3 Deployment lépések.....	12
7.4 Monitoring és logok.....	12
8. Biztonság	12
9. Adatszinkronizálási folyamat (tipikus eset)	13
10. Lehetséges jövőbeli bővítések.....	13
11. PC-s alkalmazás lehetősége	13

1. Áttekintő architektúra

A rendszer klasszikus háromrétegű architektúrát követ:

- Kliensoldal (Frontend): Vue 3 alapú egyoldalas alkalmazás (SPA), webes (frontend/en-beta/) és Capacitor mobil (iOS, Android – mobil/) klienssel, közös view- és composable-szerkezettel
- Szerveroldal (Backend / API): Laravel 12 REST API + Laravel Reverb WebSocket szerver
- Adatbázis réteg: MySQL / MariaDB relációs adatbázis, minden mérés és felhasználói adat tárolására

A kommunikáció a kliens és a szerver között JSON formátumú HTTP/HTTPS üzenetekkel történik, valós idejű frissítések a Pusher protokollt használó WebSocket csatornán keresztül érkeznek. A mobil kliens a natív egészségügyi aggregátorokhoz (HealthKit, Health Connect) a cordova-plugin-health Capacitor-kompatibilis pluginen keresztül csatlakozik.

A /mobil/ a mobil fejlesztési terület; a /frontend/ a webes verzió.

2. Technológiai stack

2.1 Backend

Komponens	Technológia	Verzió
Keretrendszer	Laravel	12.x
API autentikáció	Laravel Sanctum	4.x
Valós idejű kommunikáció	Laravel Reverb	1.x
Adatbázis	MySQL / MariaDB	8.x / 10.x
E-mail kiszolgáló	Mailtrap SMTP	3.10
Migráció + seed	Laravel Eloquent migrations	–

2.2 Frontend (web + mobil közös kódbázis)

Komponens	Technológia	Verzió
UI framework	Vue	3.5
Router	vue-router	5.x (Vue 3-kompatibilis fork)
Lokalizáció	vue-i18n	11.3
HTTP(S) kliens	axios	1.13
WebSocket	laravel-echo + pusher-js	2.3 / 8.4
Grafikonok	Chart.js	4.5
UI primitívek	reka-ui	2.9
Ikonok	lucide-vue-next	0.564
Stílus	Tailwind CSS	4.1
Build tool	Vite	7.x
Reaktív segédek	@vueuse/core	14.x

2.3 Mobil shell és pluginek (csak mobil/)

Komponens	Technológia	Verzió
Mobil shell	Capacitor	8.x
Health integráció	cordova-plugin-health	3.2
Helyi értesítés	@capacitor/local-notifications	8.x
Hálózati állapot	@capacitor/network	8.x
App lifecycle	@capacitor/app	8.x
Haptics	@capacitor/haptics	8.x
Asset generátor	@capacitor/assets	3.0

3. Backend (REST API)

3.1 Általános jellemzők

- RESTful architektúra, JSON kérés- / válasz formátum
- Laravel Sanctum személyes hozzáférési token (Bearer) autentikáció
- CORS konfigurálva a web és mobil domainekre
- Minden védett route az auth:sanctum middleware mögött
- Admin route-ok külön admin middleware-rel védettek

3.2 Végpontok (api.php)

Publikus (nem védett)

Metódus	Útvonal	Leírás
POST	/register	Új felhasználó regisztráció (throttle 10/perc)
POST	/login	Bejelentkezés, token kiadás (throttle 10/perc)
POST	/forgot-password	Jelszó-visszaállító link küldése (throttle 5/perc)
POST	/reset-password	Jelszó visszaállítása tokennel (throttle 5/perc)
GET	/email/verify/{id}/{hash}	E-mail megerősítő signed link

Védett (auth:sanctum)

Metódus	Útvonal	Leírás
GET	/heart-rates	Pulzusbejegyzések listája
GET	/average/heart-rates	Pulzus átlagok
POST	/new-heart-rate	Új pulzus mentése
PUT	/heart-rates/{id}	Pulzus szerkesztése
DELETE	/heart-rates/{id}	Pulzus törlése
GET	/blood-pressures	Vérnyomás bejegyzések
GET	/average/blood-pressures	Vérnyomás átlagok
POST	/new-blood-pressure	Új vérnyomás mentése
PUT	/blood-pressures/{id}	Vérnyomás szerkesztés
DELETE	/blood-pressures/{id}	Vérnyomás törlése
GET	/weights	Súlybejegyzések
GET	/current-weight	Legutóbbi súly
POST	/new-weight	Új súly mentése
PUT	/weights/{id}	Súly szerkesztés
DELETE	/weights/{id}	Súly törlése
GET	/calories	Kalóriabejegyzések
POST	/new-calorie	Új kalória mentése
PUT	/calories/{id}	Kalória szerkesztés
DELETE	/calories/{id}	Kalória törlése
GET	/waters	Vízbevitel bejegyzések
GET	/waters/today	Mai vízbevitel összegzése
POST	/new-water	Új vízbevitel
PUT	/waters/{id}	Vízbevitel szerkesztés
DELETE	/waters/{id}	Vízbevitel törlése
GET	/sleeps	Alvás bejegyzések
GET	/sleeps/last-night	Legutóbbi alvás rekord
POST	/new-sleep	Új alvás bejegyzés
PUT	/sleeps/{id}	Alvás szerkesztés
DELETE	/sleeps/{id}	Alvás törlése
GET	/activities	Sportaktivitás-katalógus (hu / en / de nevek)
GET	/exercises	Edzések
POST	/new-exercise	Új edzés
DELETE	/exercises/{id}	Edzés törlése
GET	/steps	Lépés előzmények
GET	/steps/today	Mai lépésszám
POST	/new-steps	Lépés rögzítése (mode: overwrite / add)
DELETE	/steps/{id}	Lépés törlése
POST	/health-sync	Batch feltöltés aggregátorból (throttle 10/perc)
POST	/device-token	Push token regisztráció

		(jelenleg LocalNotifications mellett tartalék)
DELETE	/device-token	Token törlése
GET	/token-check	Token + user + streak lekérdezés
GET	/config	Reverb kapcsolódási adatok (key, host, port, scheme)
POST	/logout	Kijelentkezés (aktuális token)
POST	/nuke	Kijelentkezés (összes token)
DELETE	/account	Teljes fiók törlése
GET	/export/json	Teljes adatexport JSON-ban
GET	/export/csv	Teljes adatexport ZIP-ben (per-tábla CSV)
GET	/streak/status	Streak állapot + freeze elérhetőség
POST	/streak/freeze	Pro: streak freeze használata (throttle 5/perc)
POST	/password-change	Jelszócsere bejelentkezett állapotban (throttle 5/perc)
POST	/profile-picture	Profilkép feltöltés
DELETE	/profile-picture	Profilkép törlés
POST	/email/verification-notification	Új verifikációs e-mail kérése
PUT	/change-plan	Free / Pro váltás
POST	/locale	Felhasználói nyelv frissítése
POST	/height	Magasság mentése (BMI-hez)
PUT	/me/goals	Napi lépés- és vízcél beállítása
PUT	/me/profile	Megjelenített név módosítása
GET	/me/progress	Felhasználói előrehaladási adatok lekérése

Metódus	Útvonal	Leírás
GET	/admin/stats	Rendszerszintű statisztikák
GET	/admin/users	Felhasználók listája (search, level, paginate)
GET	/admin/users/{user}	Felhasználó részletei
PUT	/admin/users/{user}/level	Jogosultság módosítása
DELETE	/admin/users/{user}	Felhasználó törlése

3.3 Batch feltöltési szabály

A POST /health-sync végpont az egyetlen támogatott útvonal nagy mennyiségű szenzoros adat feltöltésére. Egyenkénti upload logikát sem a HealthKit / Health Connect integrációból, sem más helyről nem szabad használni – egy korábbi 500 POST/sync a teljes appot lefagyasztotta. A batch korlátok kérelmenként:

- 500 heart_rates
- 500 blood_pressures
- 500 weights
- 500 steps
- 200 exercises

A vízbevitel és alvás nem része a batchnek (manuális forrás). A végpont egyetlen adatbázis-tranzakcióban menti a bejegyzéseket. Lépésnél napi lock + select + update / create biztosítja, hogy egy napra csak egy rekord jöjjön létre. Befejezéskor egy health_sync_complete típusú UploadCreated event mehet a usernek, és a streak csak akkor frissül, ha a batch tartalmaz mai napra szóló rekordot (batchHasToday guard).

3.4 Streak frissítés

A rendszer minden rögzített mérésnél (pulzus, vérnyomás, súly, kalória, víz, alvás, lépés) automatikusan frissíti a felhasználói streaket. Ez a logika központosított: megvizsgálja, hogy az új mérés dátuma illeszkedik-e az előzőhöz. Ha egymást követő napról van szó, növeli a streaket; ha már ugyanarra a napra történt mérés, nem változtat; ha kimaradt egy nap, a streak újraindul. A rendszer figyel arra is, hogy ne lehessen régebbi méréssel mesterségesen növelni a sorozatot, és automatikusan rögzíti az elért legnagyobb értéket.

4. Valós idejű kommunikáció (WebSocket)

4.1 Technológia

- Laravel Reverb – natív Pusher-kompatibilis szerver (külön port, prod 443/wss)
- Frontend: laravel-echo + pusher-js, az useEcho composable singleton instance-ot tart
- Protokoll: WSS (titkosított WebSocket) éles környezetben
- Csatornaszint: private (Sanctum guard a /broadcasting/auth endpointon)

A Reverb endpoint adatait a kliens nem build-time-ban, hanem futásidőben kéri a GET /config endpointról; így a host/key változás újra-deploy nélkül átállítható.

4.2 Csatornák és események

Csatorna	Guard	Esemény	Payload
private-user.{userId}	sanctum	upload.created	{ type, data } egyetlen rekordhoz, { type: "health_sync_complete", counts } batch után

A type lehetséges értékei: heart_rate, blood_pressure, weight, steps, exercise, water, sleep, health_sync_complete.

5. Frontend architektúra

5.1 Projekt struktúra

mobil/src/
App.vue – gyökér komponens
main.js – app bootstrap, router, i18n, axios interceptor
router/ – Vue Router definíció + guards
layouts/
AppShell.vue – fő layout, navigáció, WS, app lifecycle, provide()
views/ – route-onkénti komponensek
Dashboard, Diary, Stats, Summary, Exercise, Profile,
Settings, Skins, Upgrade, Admin, HealthSync, Auth,
Landing, Legal, ResetPassword, VerifyEmail
composables/ – újrafelhasználható reaktív logika
useEcho, useHealthSync, useOfflineSync, useLocalNotifications,
useSkins, useTheme, useAccess, usePullToRefresh
lib/ – keret-független segédek
api.js – axios wrapper (cache + queue)
offlineCache.js – GET cache, TTL, WS invalidation, clock guard
offlineQueue.js – POST/PUT/DELETE queue, 4xx discard
httpError.js – lokalizálható hibaüzenetek
i18n.js, utils.js, storage.js, healthUploadTracker.js, haptics.js
locales/
hu.js, en.js, de.js – fordítási kulcsok

A frontend/src/ ugyanezt a struktúrát követi tisztán webes környezetben, Capacitor-specifikus részek nélkül (useLocalNotifications, usePullToRefresh, haptics.js hiányzik). A HealthSync nézet a webes verzióban nem érhető el (a HealthSync kártya v-if="isNative"-vel rejtett).

5.2 AppShell és provide / inject

Az alkalmazás egy központi komponenssel (AppShell) figyeli a WebSocket-kapcsolatot, és a teljes alkalmazás számára elérhetővé teszi a felhasználó adatait, a kiválasztott napot, a kinézetet, a kapcsolat állapotát, valamint a legutóbbi frissítési eseményt. Amikor a szerverről új adat érkezik (pl. másik eszközről feltöltött mérés), az AppShell jelzi a megfelelő nézeteknek, hogy töltsék újra az érintett adatokat – így a felület mindig naprakész marad anélkül, hogy az egész oldalt újra kellene tölteni.

5.3 Offline kezelés

A rendszer az adatokat ideiglenesen a böngésző helyi tárhelyén tárolja, így ha nincs internetkapcsolat, a már letöltött információk továbbra is láthatók. Ha a felhasználó új adatot akar rögzíteni (például vérnyomást vagy súlyt), de éppen nincs hálózat, a kérés nem vész el: egy sorba kerül, és amint újra elérhető az internet, a rendszer automatikusan elküldi. Az érvénytelen kéréseket (például hibás formátum) a rendszer nem próbálja újra, a hálózati hibákból adódókat viszont igen. Az alkalmazás figyel arra is, hogy a privát böngészős mód ne okozzon hibát a helyi tárolás során.

6. Egészségügyi aggregátor integráció

A rendszer mobil kliense a mobil OS natív egészségügyi platformjaival a cordova-plugin-health Capacitor-pluginen keresztül kommunikál. A plugin egységes API-t biztosít a HealthKit (iOS) és a Health Connect (Android) felé.

6.1 Támogatott adattípusok

- heart_rate (pulzus)
- blood_pressure (szisztolés + diasztolés)
- weight (testsúly)
- steps (napi lépés)
- activity (edzés típusa + időtartama)

A vízbevitel és alvás nem szinkronizálódik az aggregátorból – ezeket a felhasználó manuálisan rögzíti a Diary nézeten.

6.2 Szinkronizációs folyamat

1. Az alkalmazás indulásakor ellenőrzi, hogy melyik platform (Apple Health vagy Google Fit) érhető el.
2. A felhasználó a HealthSync nézetben kifejezetten engedélyezi az adatok olvasását. Az alkalmazás ekkor lekéri az engedélyt a rendszertől, ha még nem lenne meg.
3. A letöltendő időszakot az határozza meg, hogy mikor volt az utolsó sikeres szinkron.

4. A platformról visszakapott nyers mérési rekordokat (pl. pulzus, lépés, alvás) típus szerint csoportosítja az alkalmazás, majd egyetlen kérdésben elküldi a backend /health-sync végpontjára. A különböző platformok elnevezéseit (pl. „Steps”, „HeartRate”) a szerveren tárolt tevékenység-azonosítókra képezi le.
5. A backend egy adatbázis-tranzakció keretében elmenti az összes érkező rekordot, majd health_sync_complete eseményt küld, amely tartalmazza, hogy hány új mérés került feldolgozásra.
6. Végül a kliens eltárolja az aktuális időpontot utolsó sikeres szinkronként, és megjegyzi a feltöltött rekordok azonosítóit is – így legközelebb már csak az ezután keletkező adatokat kell letöltenie.

6.3 Hibakezelés

- A pluginState állapotgép: checking → ready / unavailable / needsPermission / error
- Init hiba esetén a state nem ragad be checking-ben, hanem error-ra vált
- A felhasználónak language-agnostic (alap angol) hibaüzenet jelenik meg, a frontend httpError.js lokalizálja a státuszkód alapján

7. Üzemeltetés és deployment

7.1 Környezetek

Környezet	URL	Cél
Production API	https://api.welfarebuddy.hu	Live REST kiszolgáló
Production WS	wss://ws.welfarebuddy.hu	Live Reverb WebSocket
Web frontend	https://(www.)welfarebuddy.hu	Web SPA
Mobil	Capacitor build iOS + Android	Alkalmazás-csomag

7.2 Tárhely és háttér szolgáltatások

- Web kiszolgáló: Apache / Nginx + PHP-FPM
- Adatbázis: MySQL 8 / MariaDB 10
- WebSocket: Laravel Reverb
- Fájltárolás: Laravel storage (public disk) – profilképekhez
- Levelezés: Mailtrap SMTP

7.3 Deployment lépések

```
# 1. Backend
git pull
cd backend
composer install --no-dev --optimize-autoloader
# .env szerkesztése: APP_URL, DB_*, MAIL_*, REVERB_*
php artisan migrate      # új migrációk futtatása
php artisan config:cache
php artisan route:cache
# php-fpm restart, supervisord-ben Reverb worker reload
```

```
# 2. Web frontend
cd frontend
npm install
npm run build
# dist/ feltöltése a webszerverre
```

```
# 3. Mobil
cd mobil
npm install
npm run build
npx cap sync
# iOS: Xcode build / archive / TestFlight
# Android: Android Studio gradle build / Play Console
```

7.4 Monitoring és logok

- Laravel log: storage/logs/laravel.log
- Pail futtatható fejlesztői környezetben (composer log streaming)
- Reverb log külön fájlban
- Kliens oldali hibák a httpError.js-ben lokalizálódnak és a UI-n jelennek meg

8. Biztonság

- HTTPS mindenütt (web + mobil + WS)
- Jelszó: bcrypt hash (Laravel hashed cast)
- Token: Sanctum személyes hozzáférési token, Bearer header, jelszócsere után minden token visszavonva (tokens()->delete())
- E-mail: aláírt URL-ek (signed URL), egyedi hash az email-hez
- Validáció: minden végpontban Laravel request validáció (date, integer, min/max, exists, unique)
- Fiók törlés: minden kapcsolódó rekord explicit törlése + profilkép storage-ból törlés + token revoke
- Admin védelem: külön admin middleware + route group prefix
- WebSocket: private csatorna Sanctum guard-dal (/broadcasting/auth)
- CORS: csak a támogatott domáinak (mobil app-id, web frontend domain)
- CSRF: web session nem használt, Sanctum token-based
- Throttle middleware az érzékeny endpointokon (lásd 3.1)

9. Adatszinkronizálási folyamat (tipikus eset)

1. Felhasználó bejelentkezik → kap egy Sanctum token
2. A kliens a tokenet axios Authorization header-be beállítja, a /config endpointról lehúzza a Reverb beállításokat, majd feliratkozik a private-user.{id} csatornára
3. Manuális vagy aggregátoros adat érkezik → POST /new-* (egyenkénti) vagy POST /health-sync (batch)
4. Backend: validálás → DB tranzakció → INSERT → streak update (StreakService vagy batch-szintű) → tranzakció commit
5. Backend: UploadCreated esemény dobása → Reverb broadcast a private-user.{id} csatornán
6. Kliens: useEcho fogadja az upload.created eseményt → AppShell hívja az invalidateCacheForWsEvent(type)-ot → a megfelelő cache kulcsok eldobódnak
7. Az érintett view (Dashboard, Diary, Stats, Summary) watch-eli a wsEvent-et és újratölt

10. Lehetséges jövőbeli bővítések

- Kalória- és kalória- / vízbevitel kamera- vagy barcode-alapú felismeréssel
- Természetes nyelvű beszélgetős adatrögzítés (pl. „ma 130-as volt a vérnyomásom”)
- Orvosi export PDF formátumban
- HealthKit / Health Connect víz- és alvásszinkron (jelenleg csak manuális)
- Push token alapú értesítések visszakapcsolása szervervezérelt kampányokhoz

11. PC-s alkalmazás lehetősége

A Vue 3 + Capacitor kódbázis Electron vagy Tauri keretrendszerbe becsomagolva natív asztali alkalmazássá alakítható Windows, macOS és Linux platformokra. A webes felület már most önállóan működőképes asztali böngészőben; a dedikált PC kliens jövőbeli opcionális fejlesztés.